

Unit 3.Polymorphism

- 3.1 Concepts of Polymorphism
 - 3.2 Compile time and Run time Polymorphism
 - 3.3 Overloading and Overriding
 - Concepts, difference and application
 - 3.4 Concepts of friend function
 - 3.5 Concepts of virtual function and pure virtual function
-

3.1 Concepts of Polymorphism

3.2 Compile time and Run time Polymorphism

3.3 Overloading and Overriding. Concepts, difference and application

The word "polymorphism" means one name, multiple forms. In simple terms, we can define polymorphism as the ability of a message to be displayed in more than one form. A real-life example of polymorphism is a person who can exhibit different characteristics simultaneously. For instance, a man can be a father, a husband, and an employee at the same time. Thus, the same person possesses different behaviours in different situations. This is called polymorphism. Polymorphism is considered one of the important features of Object-Oriented Programming.

In C++ polymorphism is mainly divided into two types:

1. Compile time(Static) Polymorphism - Implemented using Function and Operator Overloading.
2. Runtime(Dynamic) Polymorphism- Implemented using Virtual Functions

Types of Polymorphism

1. Compile time polymorphism: This type of polymorphism is achieved by function overloading or operator overloading.

Function Overloading When there are multiple functions with the same name but different parameters, these functions are said to be overloaded. Functions can be overloaded by changing the number of arguments and/or the type of arguments.

Example 3.1 Function overloading

```
#include <iostream.h>
#include <conio.h>

class A
{
public:
void display(int a)
{
    cout<<a<<endl;
}
void display(double b)
{
    cout<<b<<endl;
```

```

    }
    void display(char *name)
    {
        cout<<name<<endl;
    }
};
void main()
{
    A a1;
    clrscr();
    a1.display(1);
    a1.display(1.2);
    a1.display("abc");
    getch();
}

```

Output

```

1
1.2
abc

```

Operator Overloading

C++ allows us to add two variables of user-defined types using the same syntax that is applied to the basic types. This means that C++ has the ability to provide operators with a special meaning for a specific data type. This mechanism of giving a special meaning to an operator for a user-defined type is known as operator overloading.

Defining Operator Overloading:

To define an additional task for an operator, we must specify what it means in relation to the class to which the operator is applied. This is done with the help of a special function called an operator function.

General form of operator function is:

```

Return type classname : : operator op(argument list)
{
    Function body
}

```

Where return type is the type of value returned by function and op is the operator being overloaded. The op is preceded by the keyword operator. operator op is the function name.

Operator function must be either member function (non static) or friend functions.

The process of overloading involves the following steps:

1. Create a class that defines the data type that is to be used in the overloading operation.
2. Declare the operator function operator op () in the public part of the class. It may be either member or friend function.
3. Define the operator function.

Overloaded operator function can be call by expressions such as

op x or x op for unary operator. E.g. $-x$ or $x-$ where x is the object of class.

x op y for binary operator. E.g. $x + y$ or $x * y$ or $x-y$ where x and y are objects of class.

Expression **x op y** would be interpreted as either

- **x . operator op(y)** in case of member function.
- **operator op(x, y)** in case of friend function.

Expression **op x or x op** would be interpreted as either

- **x . operator op()** in case of member function.
- **operator op(x)** in case of friend function.

Rules for overloading operators

1. Only existing operators can be overloaded. New operator cannot be created.
2. The overloaded operator must have at least one operand that is of user defined type.
3. We cannot change the basic meaning of an operator. We cannot redefine the Plus (+) operator to subtract (-) one value from another.
4. Overloaded operators follow the syntax rules of the original operators. They cannot be overridden.
5. There are some operators that cannot be overloaded.

Sizeof	Size of operator
.	Membership operator
.*	Pointer to member operator
::	Scope resolution operator
?:	Conditional operator
6. We cannot use the friend function to overload certain operators. However member functions can be used to overload them.

=	assignment operator
()	function call operator
[]	subscripting operator
->	class member access operator
7. Unary operator overloaded by member function, take no explicit arguments and return no explicit values. But those overloaded by a friend function take one explicit argument (object of the relevant class).
8. Binary operator overloaded by member function, take one explicit argument and those overloaded by a friend function take two explicit arguments.
9. When using a binary operator overloaded through a member function the left hand operand must be an object of the relevant class.
10. Binary arithmetic operators such as $+$, $-$, $*$, $/$ must explicitly return a value.

Example 3.2 Program for Unary (-) and Binary (+) Operator Overloading Using a Member Function

```
#include <iostream.h>
#include <conio.h>
```

```
class data
{
```

```

private:
    int a,b;
public:
    data()
    {
        a=0;
        b=0;
    }
    data(int x,int y)
    {
        a=x;
        b=y;
    }
    data operator+(data d2)//binary operator +
    {
        data d;
        d.a=a+d2.a;
        d.b=b+d2.b;
        return(d);
    }
    void operator-()//unary - operator
    {
        a=-a;
        b=-b;
    }
    void display()
    {
        cout<<"a= "<<a<<endl<<"b= "<<b<<endl;
    }
};

void main()
{
    clrscr();
    data d1(-1,-1),d2(2,2),d3;
    d1.operator-();
    //-d1;
    d1.display();
    d3=d1.operator+(d2);
    //d3=d1+d2;
    d3.display();
    getch();
}

```

Output

```

1
1
3
3

```

Example 3.3 Program for Unary (-) and Binary (+) Operator Overloading Using a Friend Function

```

#include <iostream.h>
#include <conio.h>
#include <string.h>
class data
{
    private:
        int a,b;
    public:
        data()
        {
            a=0;
            b=0;
        }
        data(int x,int y)
        {
            a=x;
            b=y;
        }

        friend data operator+(data d1,data d2)
        {
            data d;
            d.a=d1.a+d2.a;
            d.b=d1.b+d2.b;
            return(d);
        }
        friend void operator-(data &d1)
        {
            d1.a=-d1.a;
            d1.b=-d1.b;
        }
        void display()
        {
            cout<<a<<b<<endl;
        }
};

void main()
{
    clrscr();
    data d3,d1(-1,-1),d2(2,2);
    operator-(d1);
    //-d1;
    d1.display();
    d2.display();
    d3=operator+(d1,d2);

```

```
//d3=d1+d2;
d3.display();
getch();
}
```

Output

```
11
22
33
```

3.4 Concepts of friend function

According to the data hiding concept in C++, no one has permission to access private and protected members from outside the class. If a developer needs access to these private or protected members, they can use friend functions. A friend function provides the privilege to access all the private and protected members of the class.

Friend Function Characteristics

1. It should be declared inside the class with the 'friend' keyword.
2. It can be defined anywhere in the program and outside of the class like a normal function, but the friend keyword must not be used in the function definition.
3. It is not within the class scope, so it cannot be called by objects of the class.
4. It can be called like a normal function.
5. Usually, it has class objects as arguments.
6. It cannot access member variables or functions directly; it can only access them with the help of the class objects using the dot membership operator.
7. It can be declared with any access specifier, and the access specifier does not impact the friend function.

Example 3.4 Write a program to find the mean of numbers using a friend function.

```
#include<iostream.h>
#include<conio.h>
class A
{
public:
    A()
    {
        cout<<"enter a and b";
        cin>>a>>b;
    }
private:
    int a,b;
    friend float mean(A);
```

```

};

float mean(A a1)
{
    return float(a1.a+a1.b)/2.0;
}
void main()
{
    clrscr();
    A a1;
    cout<<"mean is"<<mean(a1);
    getch();
}
Output
enter a and b1 2
mean is1.5

```

Passing and returning object

In C++ we can pass class's objects as arguments and also return them from a function the same way we pass and return other variables.

Syntax `object_type function_name(object);`

Example: Write a program to add two time objects and display the sum using the friend function.

```

//object as argument and return
//read and add time
#include<iostream.h>
#include<conio.h>

class time
{
    int hours;
    int min;
public:
    void gettime(int h,int m)
    {
        hours=h;
        min=m;
    }
    void puttime()
    {
        cout<<"\n\n\t Hours :- "<<hours;
        cout<<"\n\n\t Minutes :- "<<min;
    }
    friend time sum(time,time);
}

```

```

};
time sum(time t1,time t2)
{
    time t;
    t.min=t1.min+t2.min;
    t.hours=t.min/60;
    t.min=t.min%60;
    t.hours=t.hours+t1.hours+t2.hours;
    return(t);
}

void main()
{
    time t1,t2,t3;
    clrscr();
    t1.gettime(1,40);
    t2.gettime(1,30);
    t3=sum(t1,t2);
    cout<<"\t\t\t TIME T1 :- ";t1.puttime();
    cout<<"\n\t\t\t TIME T2 :- ";t2.puttime();
    cout<<"\n\t\t\t THE ADDITION OF TWO TIMES ";
    cout<<"\n\n\t\t\t TIME T3 :- ";t3.puttime();
    getch();
}

```

Output:

```

                                TIME T1:-
Hours:- 1
Minutes:- 40

                                TIME T2:-
Hours:- 1
Minutes:- 30

                                THE ADDITION OF TWO TIMES
                                TIME T3:-
Hours:- 3
Minutes:- 10

```

3.5 Concepts of virtual function and pure virtual function

A virtual function is a member function declared within a base class and redefined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function. Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for the function call. They are mainly used to achieve runtime polymorphism. Functions are declared with the virtual keyword in the base class. The resolving of function calls is done at runtime.

Rules for Virtual Functions:

1. Virtual functions cannot be static.
2. A virtual function can be a friend function of another class.
3. Virtual functions should be accessed using a pointer or reference of the base class type to achieve runtime polymorphism.
4. The prototype of virtual functions should be the same in the base as well as derived class.
5. They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override (or redefine) the virtual function; in that case, the base class version of the function is used.
6. A class may have a virtual destructor, but it cannot have a virtual constructor.

Example 3.5 virtual function

```
#include <iostream.h>
#include <conio.h>

class base
{
public:
virtual void print()
{
cout << "print base class" << endl;
}
void show()
{
cout << "show base class" << endl;
}
};

class derived : public base
{
public:
void print()
{
cout << "print derived class" << endl;
}
void show()
{
cout << "show derived class" << endl;
}
};

int main()
{
base *bptr;
derived d;
bptr = &d;
clrscr();
// virtual function, binded at runtime
bptr->print();
// Non-virtual function, binded at compile time
bptr->show();
```

```

getch();
return(0);
}

```

Output

print derived class

show base class

A virtual function is a member function declared within a base class and is redefined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.

Example virtual void print() {cout<<"hello";}

Pure Virtual Functions in C++:

A pure virtual function (or abstract function) in C++ is a virtual function for which we don't have an implementation; we only declare it. A pure virtual function is declared by assigning 0 in the declaration.

Example virtual void print()=0;

Programming exercises and solutions

1. Write a program to find the area of a circle and a rectangle using function overloading.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class test
```

```
{
```

```
    public:
```

```
        void area(double r)
```

```
        {
```

```
            cout<<"area of circle is "<<3.14*r*r<<endl;
```

```
        }
```

```
        void area(float l,float b)
```

```
        {
```

```
            cout<<"area of rect is "<<l*b<<endl;
```

```
        }
```

```
};
```

```
void main()
```

```
{
```

```
    test t;
```

```
    clrscr();
```

```
    t.area(10);
```

```
    t.area(1.5,1.5);
```

```
    getch();
```

```
}
```

2. operator overload + to concat two strings and == to compare strings.

```
#include <iostream.h>
#include <conio.h>
#include <string.h>
class data
{
    private:
        char *a;
    public:
        data()
        {
            a=0;//null string;
        }
        data(char *s)
        {
            strcpy(a,s);
        }

        data operator+(data d2)
        {
            data d3;
            strcpy(d3.a,a);
            return(strcat(d3.a,d2.a));
        }
        void show()
        {
            cout<<a;
        }
        int operator==(data d2)
        {
            if(strcmp(a,d2.a)==0)
                return(1);
            else
                return(0);
        }
};

void main()
{
    clrscr();
    char s1[10],s2[10];
    cin>>s1>>s2;
    data d1(s1),d2(s2),d3;
    //d3=d1.operator+(d2);
    d3=d1+d2;
    d3.show();
    if(d1==d2)
        cout<<"\n same";
```

```

else
    cout<<"\n not same";
    getch();
}

```

3.operator overload > to find a large number from two numbers.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
#include <string.h>
```

```
class data
```

```
{
```

```
private:
```

```
int a;
```

```
public:
```

```
data()
```

```
{
```

```
    cin>>a;
```

```
}
```

```
int operator>(data d2)
```

```
{
```

```
    if(a>d2.a)
```

```
        return(1);
```

```
    else
```

```
        return(0);
```

```
}
```

```
void show()
```

```
{
```

```
    cout<<a;
```

```
}
```

```
};
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    data d1,d2;
```

```
    if(d1>d2)
```

```
{
```

```
        d1.show();
```

```
        cout<<" is large";
```

```
}
```

```
else
```

```
{
```

```
        d2.show();
```

```
        cout<<" is large";
```

```
}
```

```
    getch();
```

```
}
```

4. Write a program to overload + operator for one dim matrix.

```
#include <iostream.h>
#include <conio.h>

class data
{
    private:
        int a[5];
    public:
        void get();
        data operator+(data);
        void display();
};

void data::get()
{
    for(int i=0;i<5;i++)
        cin>>a[i];
}

void data:: display()
{
    for(int i=0;i<5;i++)
        cout<<a[i]<<endl;
}

data data::operator+(data d2)
{
    data d;
    for(int i=0;i<5;i++)
        d.a[i]=a[i]+d2.a[i];
    return(d);
}

void main()
{
    clrscr();
    data d,d1,d2;
    d1.get();
    d2.get();
    clrscr();
    d=d1+d2;
    d.display();
    getch();
}
```

5. Create a base class shape, declare two data members dim1 and dim2 and declare- calculate area function as virtual function. Derive two class rectangle and triangle from the shape class that override calculate area function.

```

#include<iostream.h>
#include<conio.h>
class shape
{
    protected:
        int d1,d2;
    public:
        virtual void area()=0;
};
class rect:public shape
{
    public:
        rect(int x,int y)
        {
            d1=x;
            d2=y;
        }

        void area()
        {
            cout<<"\n area of rectangle"<<d1*d2;
        }
};
class triangle:public shape
{
    public:
        triangle(int x,int y)
        {
            d1=x;
            d2=y;
        }

        void area()
        {
            cout<<"\n area of triangle "<<0.5*d1*d2;
        }
};

void main()
{
    clrscr();
    shape *s1;
    rect r(10,10);
    triangle t(20,20);
    s1=&r;
    s1->area();
    s1=&t;
    s1->area();
    getch();
}

```

```
}

```

Output

```
area of rectangle 100

```

```
area of triangle 200

```

6. Write a program to create a base class media with data member (title, price). derive tape and book from the media. Declare virtual function display in media and display book detail (title, price, page) and tape detail (title, price, playtime) using display function in respective classes.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
class media
{
    protected:
    char title[50];
    float price;
    public:
    media(char *s, float a)
    {
        strcpy(title, s);
        price = a;
    }

    virtual void display() = 0;
};

class tape : public media {
public:
    float time;
    tape(char *s, float a, float t) : media(s, a)
    {
        time = t;
    }
    void display()
    {
        cout << "\n Title" << title;
        cout << "\n Prices" << price;
        cout << "\n Play time" << time;
    }
};

class book : public media {
public:
    int pages;
    book(char *s, float a, int p) : media(s, a)

```

```

        {
            pages=p;
        }
void display()
{
    cout << "\n Title"<<title;
    cout << "\n Prices"<<price;
    cout << "\n Pages"<<pages;
}
};
void main()
{
    char *t;
    float p,pt;
    int pg;

    clrscr();
    //Book detail
    cout<<"\n enter book detail\n";
    cout<<"Title ";cin>>t;
    cout<<"Prices ";cin>>p;
    cout<<"Pages ";cin>>pg;

    book bk(t,p,pg);

    //Tape detail
    cout<<"\n enter tape detail\n";
    cout<<"Title ";cin>>t;
    cout<<"Prices ";cin>>p;
    cout<<"Play time ";cin>>pt;

    tape tp(t,p,pt);

    clrscr();
    media *list[2];
    list[0]=&bk;
    list[1]=&tp;

    cout<<"\n Book Detail";
    list[0]->display(); // call book display fun

    cout<<"\n\n Tape Detail";
    list[1]->display();

    getch();
}

```